

Week 8 -Ch05. 회귀

01 회귀 소개

- 사람의 키는 평균 키로 회귀하려는 경향을 가진다는 자연의 법칙이 있다
→ 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법
- 여러 개의 독립변수와 한 개의 종속변수 간의 상관관계를 모델링하는 기법 통칭
 - e.g.) 아파트의 방 개수, 방 크기, 주변 학군 등 여러 개의 독립 변수에 따라 아파트 가격이라는 종속변수가 어떤 관계?
→ 모델링 및 예측
 - $Y = W1*X1 + W2*X2 + W3*X3 + \dots + Wn*Xn$ 이라는 선형 회귀식
 - Y는 종속변수, 아파트 가격
 - X들은 독립변수, 각각 방 개수, 방 크기, 주변 학군 등
 - W들은 회귀 계수(Regression Coefficients), 독립변수의 값에 영향 미치는 수
- 머신러닝 관점에서 독립변수는 feature에 해당
- 종속변수는 결정 값
- 머신러닝 회귀 예측의 핵심은 주어진 feature와 결정 값 데이터 기반에서 학습을 통해 최적의 회귀 계수 찾는 것
- 회귀의 유형 기준
 - 선형/비선형 여부
 - 독립변수의 개수
 - 종속변수의 개수
- 회귀에서 가장 중요한 것은 회귀 계수
 - | 독립변수 개수 | 회귀 계수 결합 |
|-------------|-------------|
| 1개: 단일 회귀 | 선형: 선형 회귀 |
| 여러 개: 다중 회귀 | 비선형: 비선형 회귀 |
- 지도 학습
 - 분류
 - 예측값이 카테고리과 같은 이산형 클래스 값

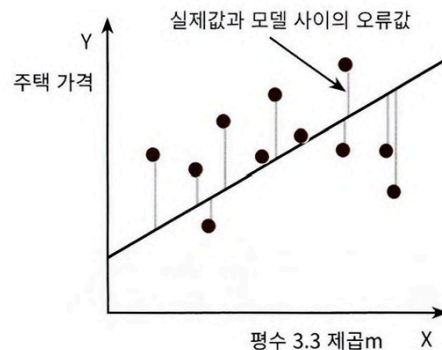
- 회귀
 - 연속형 숫자 값
- 선형 회귀가 회귀 중 가장 많이 사용 됨
 - 실제 값과 예측값의 차이를 최소화하는 직선형 회귀선을 최적화하는 방식
 - Regularization 방법에 따라 다시 별도의 유형으로 나눌 수 있음
 - Regularization(규제)는 일반적인 선형 회귀의 과적합 해결하기 위해 회귀 계수에 패널티 값 적용하는 것
- 대표적인 선형 회귀 모델
 - 일반 선형 회귀: 예측값과 실제 값의 RSS(Residual Sum of Squares)를 최소화할 수 있도록 회귀 계수를 최적화하며, 규제(Regularization)를 적용하지 않은 모델입니다.
 - 릿지(Ridge): 릿지 회귀는 선형 회귀에 L2 규제를 추가한 회귀 모델입니다. 릿지 회귀는 L2 규제를 적용하는데, L2 규제는 상대적으로 큰 회귀 계수 값의 예측 영향도를 감소시키기 위해서 회귀 계수값을 더 작게 만드는 규제 모델입니다.
 - 라쏘(Lasso): 라쏘 회귀는 선형 회귀에 L1 규제를 적용한 방식입니다. L2 규제가 회귀 계수 값의 크기를 줄이는 데 반해, L1 규제는 예측 영향력이 작은 피처의 회귀 계수를 0으로 만들어 회귀 예측 시 피처가 선택되지 않게 하는 것입니다. 이러한 특성 때문에 L1 규제는 피처 선택 기능으로도 불립니다.
 - 엘라스틱넷(ElasticNet): L2, L1 규제를 함께 결합한 모델입니다. 주로 피처가 많은 데이터 세트에서 적용되며, L1 규제에 피처의 개수를 줄임과 동시에 L2 규제에 계수 값의 크기를 조정합니다.
 - 로지스틱 회귀(Logistic Regression): 로지스틱 회귀는 회귀라는 이름이 붙어 있지만, 사실은 분류에 사용되는 선형 모델입니다. 로지스틱 회귀는 매우 강력한 분류 알고리즘입니다. 일반적으로 이진 분류뿐만 아니라 희소 영역의 분류, 예를 들어 텍스트 분류와 같은 영역에서 뛰어난 예측 성능을 보입니다.

02 단순 선형 회귀를 통한 회귀 이해

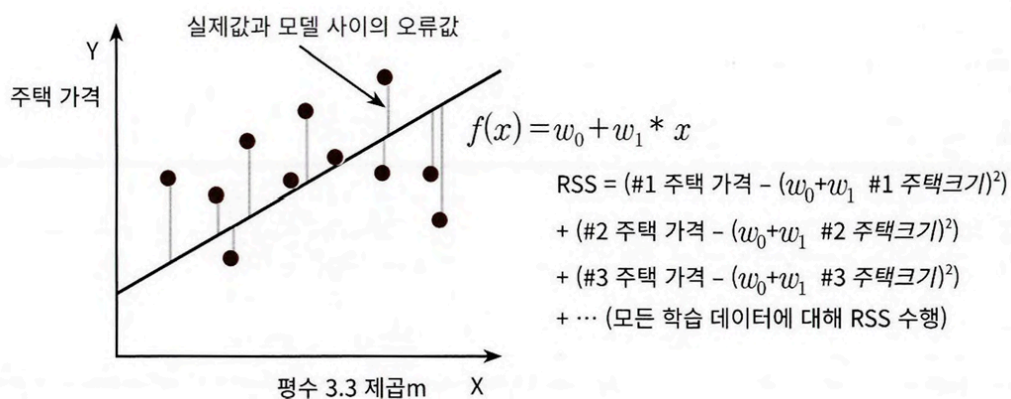
단순 선형 회귀

- 독립변수도 하나, 종속변수도 하나인 선형 회귀
- e.g.) 주택 가격이 주택 크기로만 결정된다면 가격은 크기에 대해 선형의 관계로 표현 가능
- 특정 기울기와 절편을 가진 1차 함수식으로 모델링 가능
- $\hat{Y} = w_0 + w_1 * X$ 로 계산 가능

- 독립변수가 1개인 단순 선형 회귀에서는 기울기 w_1 과 절편 0 을 회귀 계수로 지칭
- 이때 실제 주택 가격은 실제 값만큼의 오류를 뺀 $w_0 + w_1 * X + \text{오류 값}$
- 실제 값과 회귀 모델의 차이에 따른 오류 값을 남은 오류, 잔차라고 함
- 최적의 회귀 모델 만든다 = 전체 데이터의 잔차 합이 최소가 되는 모델을 만든다는 의미
- 동시에 오류 값 합이 최소가 될 수 있는 최적의 회귀 계수 찾는다는 의미



- 오류 값은 +나 -가 될 수 있음
- 그래서 전체 데이터의 오류 합을 구하기 위해 단순히 더했다가는 뜻하지 않게 오히려 오류 합 크게 줄어 들 수 있음
- ⇒ 보통 오류 합 계산할 때는
 - MAE(Mean Absolute Error)
 - RSS(Residual Sum of Square)을 취함
- 일반적으로 미분 등의 계산의 편리성 위해 RSS 방식으로 구함
 - ⇒ $\text{Error}^2 = \text{RSS}$



- RSS는 변수가 w_0, w_1 인 식으로 표현 가능
- RSS를 최소로 하는 w_0, w_1 찾는 것이 머신러닝 기반 회귀의 핵심 사항
- RSS는 회귀식의 독립변수 X , 종속변수 Y 사 중심 변수가 아니라 w 변수(회귀 계수)가 중심 변수임을 인지하는 것이 매우 중요
 - 학습 데이터로 입력되는 독립변수와 종속변수는 RSS에서 모두 상수로 간주
- 일반적으로 RSS는 학습 데이터의 건수로 나누어서 정규화된 식으로 표현

$$RSS(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

(i는 1부터 학습 데이터의 총 건수 N 까지)

- 회귀에서 이 RSS는 비용(Cost)
- w 변수(회귀계수)로 구성되는 RSS를 비용 함수라고 함
- 머신러닝 회귀 알고리즘은 데이터를 계속 학습하면서
 - 이 비용 함수가 반환하는 값 = 오류 값을 지속해서 감소시키기
 - 최종적으로 더 이상 감소하지 않는 최소의 오류 값 구하는 것
- 비용 함수 = 손실 함수(loss function)

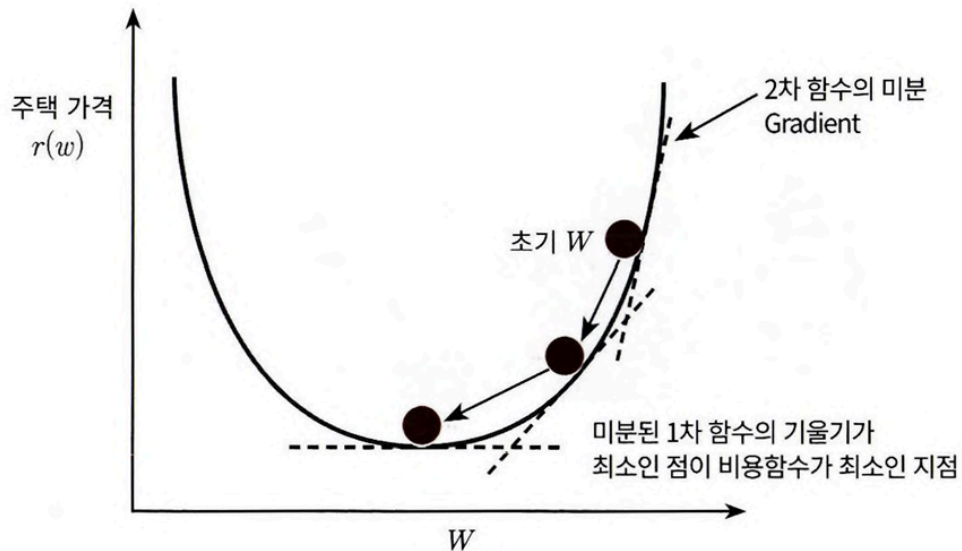
03 비용 최소화하기 - 경사 하강법(Gradient Descent) 소개

어떻게 비용 함수 최소 되는 W 파라미터 구함?

경사 하강법

- 고차원 방전식에 대한 문제를 해결
- 비용 함수 RSS를 최소화하는 방법을 직관적으로 제공
- '데이터 기반으로 알고리즘이 스스로 학습한다'는 머신러닝 개념을 가능하게 만들어준 핵심 기법
- '점진적으로' 반복적인 계산을 통해 W 파라미터 값을 업데이트하면서 오류 값이 최소가 되는 W 파라미터 구하는 방식
- '어떻게 하면 오류가 작아지는 방향으로 W 값을 보정할 수 있을까?'
- 최초 w 에서부터 미분 적용, 미분 값 감소하는 방향으로 순차적으로 w 업데이트

- 더 이상 미분 값 감소하지 않는 지점: 비용 함수가 최소인 지점
- 그때의 w 를 반환



- $R(w)$ 를 최소화하는 w_0 과 w_1 의 값은 각각 $r(w)$ 를 w_1, w_0 으로 순차적으로 편미분으로 얻음

$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N -x_i * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$$

$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N} \sum_{i=1}^N -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$$

- 이때 편미분 값이 너무 클 수 있기 때문에 보정 계수 α 곱함
 - 이때 α = 학습률(learning rate)
 - Step 1: w_1, w_0 를 임의의 값으로 설정하고 첫 비용 함수의 값을 계산합니다.
 - Step 2: w_1 을 $w_1 + \eta \frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$, w_0 을 $w_0 + \eta \frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$ 으로 업데이트한 후 다시 비용 함수의 값을 계산합니다.
 - Step 3: 비용 함수가 감소하는 방향으로 주어진 횟수만큼 Step 2를 반복하면서 w_1 과 w_0 를 계속 업데이트합니다.

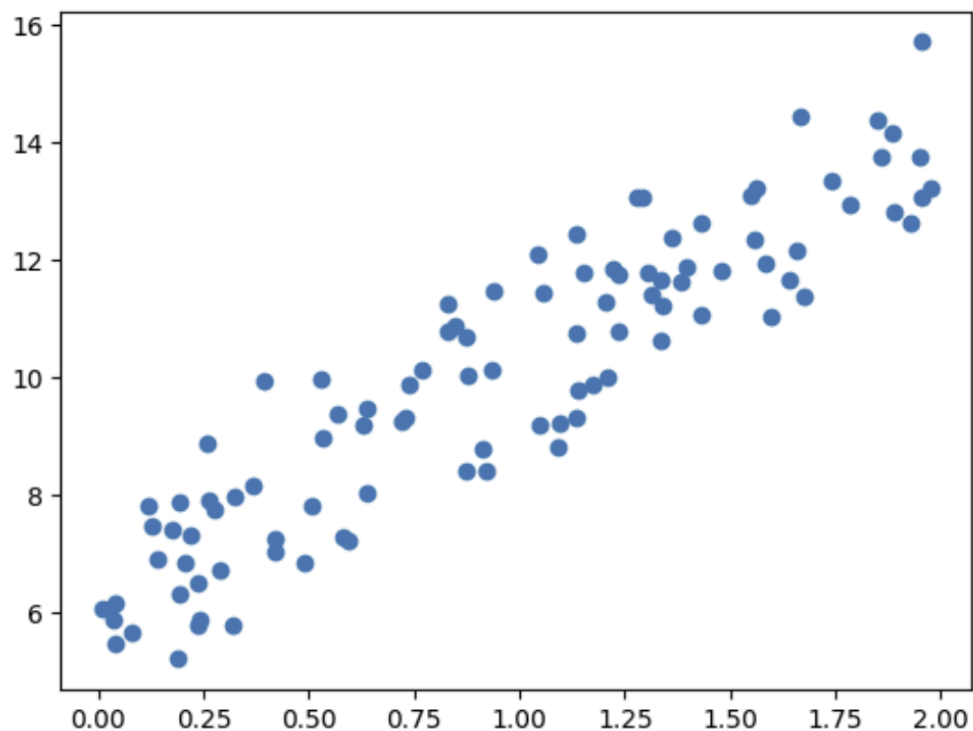
$y=4x+6$ 근사하기

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

np.random.seed(0)
# y = 4X + 6 식을 근사(w1=4, w0=6). random 값은 Noise를 위해 만듦
X = 2 * np.random.rand(100,1)
y = 6 + 4 * X + np.random.randn(100,1)

# X, y 데이터 셋 scatter plot으로 시각화
plt.scatter(X, y)
```

<matplotlib.collections.PathCollection at 0x7a90eada7ed0>



```
def get_cost(y, y_pred):
    N = len(y)
    cost = np.sum(np.square(y-y_pred))/N
    return cost
```

- gradient_descent() 함수 생성, 구현
- w1, w0 모두 0으로 초기화한 뒤 iters 개수만큼 반복하며 업데이트
- get_weight_updates() 함수에서 입력 배열 X값에 대한 예측 배열 y_pred는 $\text{np.dot}(X, w1.T) + w0$ 으로 구함
- 입력 배열 X와 w1 배열이 내적과 동일

```

# w1 과 w0 를 업데이트 할 w1_update, w0_update를 반환.
def get_weight_updates(w1, w0, X, y, learning_rate=0.01):
    N = len(y)
    # 먼저 w1_update, w0_update를 각각 w1, w0의 shape와 동일한 크기를 가진 0 값으로 초기화
    w1_update = np.zeros_like(w1)
    w0_update = np.zeros_like(w0)
    # 예측 배열 계산하고 예측과 실제 값의 차이 계산
    y_pred = np.dot(X, w1.T) + w0
    diff = y - y_pred

    # w0_update를 dot 행렬 연산으로 구하기 위해 모두 1값을 가진 행렬 생성
    w0_factors = np.ones((N,1))

    # w1과 w0을 업데이트할 w1_update와 w0_update 계산
    w1_update = -(2/N)*learning_rate*(np.dot(X.T, diff))
    w0_update = -(2/N)*learning_rate*(np.dot(w0_factors.T, diff))

    return w1_update, w0_update

```

```

# 입력 인자 iters로 주어진 횟수만큼 반복적으로 w1과 w0를 업데이트 적용함.
def gradient_descent_steps(X, y, iters=10000):
    # w0와 w1을 모두 0으로 초기화.
    w0 = np.zeros((1,1))
    w1 = np.zeros((1,1))

    # 인자로 주어진 iters 만큼 반복적으로 get_weight_updates() 호출하여 w1, w0 업데이트 수행
    for ind in range(iters):
        w1_update, w0_update = get_weight_updates(w1, w0, X, y, learning_rate=0.01)
        w1 = w1 - w1_update
        w0 = w0 - w0_update

    return w1, w0

```

```

def get_cost(y, y_pred):
    N = len(y)
    cost = np.sum(np.square(y - y_pred))/N
    return cost

w1, w0 = gradient_descent_steps(X, y, iters=1000)
print("w1:{0:.3f} w0:{1:.3f}".format(w1[0,0], w0[0,0]))
y_pred = w1[0,0] * X + w0
print('Gradient Descent Total Cost:{0:.4f}'.format(get_cost(y, y_pred)))

```

```

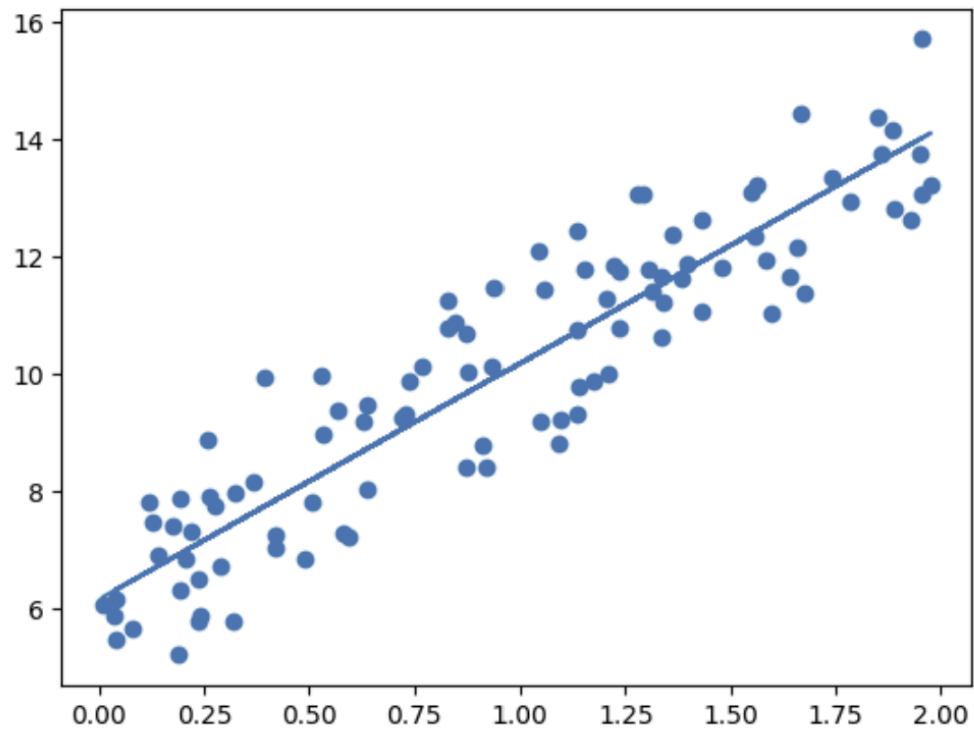
w1:4.022 w0:6.162
Gradient Descent Total Cost:0.9935

```

- 실제 선형식인 $y=4x+6$ 과 유사하게 $w1=4.022$, $w0=6.162$ 도출
- 예측 오류 비용= 약 0.9935
- y_pred 에 기반한 회귀선

```
plt.scatter(X,y)
plt.plot(X, y_pred)
```

[<matplotlib.lines.Line2D at 0x7a90c482e410>]



- 일반적으로 경사하강법은 모든 학습 데이터에 대해 반복적으로 값 업데이트
→ 매우 오래 걸림
⇒ 실전에서는 확률적 경사 하강법(Stochastic Gradient Descent) 사용

```
def stochastic_gradient_descent_steps(X, y, batch_size=10, iters=1000):
    w0 = np.zeros((1, 1))
    w1 = np.zeros((1, 1))
    for ind in range(iters):
        np.random.seed(ind)
        # 전체 X, y 데이터에서 랜덤하게 batch_size만큼 데이터 추출하여 sample_X, sample_y로
        stochastic_random_index = np.random.permutation(X.shape[0])
        sample_X = X[stochastic_random_index[0:batch_size]]
        sample_y = y[stochastic_random_index[0:batch_size]]
        # 랜덤하게 batch_size만큼 추출된 데이터 기반으로 w1_update, w0_update 계산 후 업데이트
        w1_update, w0_update = get_weight_updates(w1, w0, sample_X, sample_y, learning_rate)
        w1 = w1 - w1_update
        w0 = w0 - w0_update

    return w1, w0
```

```
w1, w0 = stochastic_gradient_descent_steps(X, y, iters=1000)
print("w1:", round(w1[0,0],3), "w0:", round(w0[0,0],3))
y_pred = w1[0,0] * X + w0
print('Stochastic Gradient Descent Total Cost:{0:.4f}'.format(get_cost(y, y_pred)))
```

w1: 4.028 w0: 6.156
Stochastic Gradient Descent Total Cost:0.9937

- 전체 입력 데이터로 w 업데이트 값 계산 X
- 일부 데이터만 이용
 - 빠른 속도 보장
- 경사 하강법으로 구한 예측 오류와 큰 성능상 차이 없음
- → 일반적으로 확률적 경사 하강법 이용
- feature가 여러 개인 경우도 1개인 경우를 확장해 유사하게 도출 가능

$$\hat{Y} = X_{mat} \underset{\text{내적}}{*} [w_1 \ w_2 \ \cdots \ w_m]^T + w_0$$

X_{mat} matrix structure:

	Feature 1	Feature 2	...	Feature M
1	x_{11}	x_{12}	$\cdots$	x_{1m}
2	x_{21}	x_{22}	$\cdots$	x_{2m}
...	$\cdots$	$\cdots$	$\cdots$	$\cdots$
n	x_{n1}	x_{n2}	$\cdots$	x_{nm}

$\hat{Y}$ 1값을 가진 피쳐 추가 X_{mat}

$$\begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix} = \begin{bmatrix} \text{Feat 0} & \text{Feat 1} & \text{Feat 2} & \dots & \text{Feat M} \\ 1 & x_{11} & x_{12} & \dots & x_{1m} \\ 1 & x_{21} & x_{22} & \dots & x_{2m} \\ \dots & \dots & \dots & \dots & \dots \\ 1 & x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix} * \begin{bmatrix} w_0 & w_1 & w_2 & \dots & w_m \end{bmatrix}^T$$

w_0 을 W 배열 내에 포함
 내적

$$\hat{Y} = X_{mat} * W^T$$

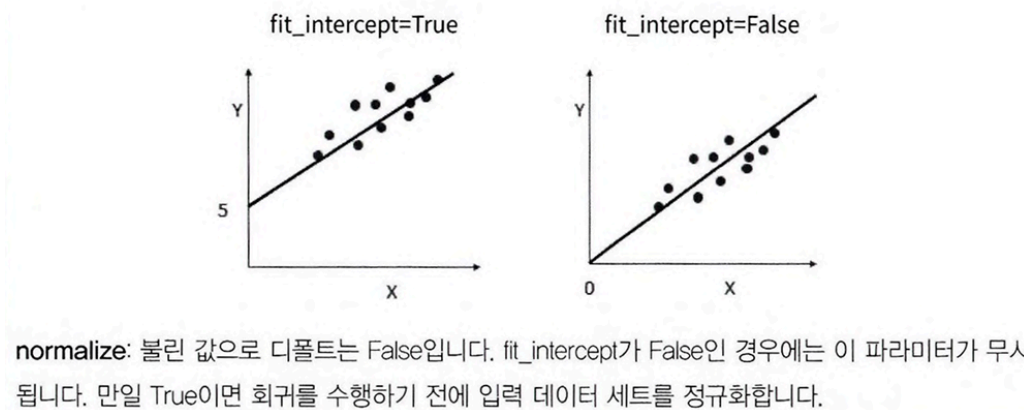
04 사이킷런 LinearRegression을 이용한 보스턴 주택 가격 예측

LinearRegression 클래스 - Ordinary Least Squares

LinearRegression 클래스

- 예측값과 실제 값의 RSS(Residual Sum of Squares)를 최소화
→ OLS(Ordinary Least Square) 추정 방식으로 구현한 클래스
- fit() 메서드로 X
- y 배열을 입력 받으면 회귀 계수(Coefficient)인 W를 coef_속성에 저장
- 입력 파라미터

fit_intercept: 불린 값으로, 디폴트는 True입니다. Intercept(절편) 값을 계산할 것인지 말지를 지정합니다. 만일 False로 지정하면 intercept가 사용되지 않고 0으로 지정됩니다.



- 속성
 - **coef_:** fit() 메서드를 수행했을 때 회귀 계수가 배열 형태로 저장하는 속성
Shape는 (Target 값 개수, feature 개수)
 - **intercept_:** intercept 값
- OLS 기반의 회귀 계수 계산은 입력 feature의 독립성에 많은 영향 받음
- feature 간의 상관관계 매우 높은 경우 분산이 매우 커져 오류에 많이 민감
- → 다중 공선성(multi-collinearity)라고 함

회귀 평가 지표

회귀 평가를 위한 지표

- 실제 값과 회귀 예측값 차이 값을 기반으로 한 지표가 중심
- 실제 값과 예측값의 차이를 그냥 더하면 +,-가 섞여 오류 상쇄
- → 오류의 절댓값 평균이나 제곱 or 제곱한 뒤 다시 루트 씌운 평균값 구함

평가 지표	설명	수식
MAE	Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변환해 평균한 것입니다.	$MAE = \frac{1}{n} \sum_{i=1}^n Y_i - \hat{Y}_i $
MSE	Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다.	$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$
RMSE	MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다.	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$
R ²	분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다.	$R^2 = \frac{\text{예측값 Variance}}{\text{실제값 Variance}}$

- 사이킷런은 RMSE를 제공하지 않음
- MSE에 제곱근 씌워 계산하는 함수 직접 만들어야 함
- 각 평가 방법에 대한 사이킷런의 API 및 cross_val_score나 GridSearchCV에서 평가 시 사용되는 scoring 파라미터의 적용

MAE	metrics.mean_absolute_error	'neg_mean_absolute_error'
MSE	metrics.mean_squared_error	'neg_mean_squared_error'
RMSE	metrics.mean_squared_error를 그대로 사용하되 squared 파라미터를 False로 설정.	'neg_root_mean_squared_error'
MSLE	metrics.mean_squared_log_error	'neg_mean_squared_log_error'
R ²	metrics.r2_score	'r2'

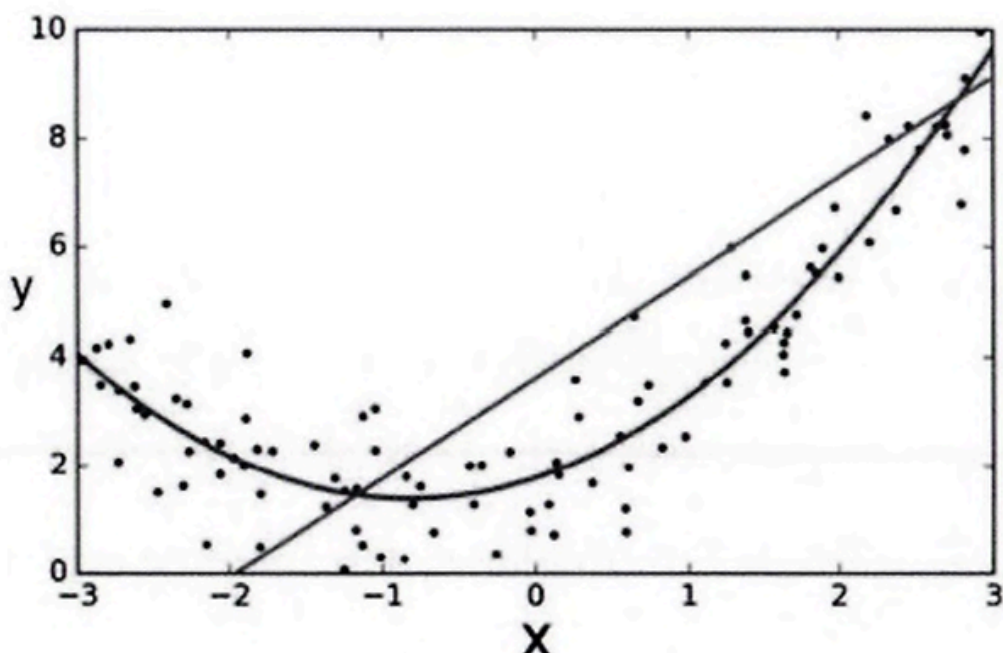
- 유의점
 - MAE scoring 파라미터 값 보면 neg_mean_absolut_error
 - neg: 음수
 - MAE는 절댓값 합이라 음수 가질 수 없음
 - 음수값 반환 이유
 - 사이킷런의 scoring함수가 score 값 클수록 좋은 평가 결과로 자동 평가
 - 실제 값과 예측 값의 오류 차이를 기반으로 하는 회귀 평가 지표의 경우 값이 커지면 오히려 나쁜 모델이라는 의미
 - ⇒ 보정 필요

- $\Rightarrow -1$ 을 원래의 평가 지표 값에 곱해서 음수 만들
- 작은 오류 값이 더 큰 숫자로 인식하게 함
- $10 > 1 \rightarrow -1 > -10$

05 다항 회귀와 과(대)적합/과소적합 이해

다항 회귀

- 회귀가 독립변수의 단항식이 아닌 다항식으로 표현되는 것
- 선형 회귀임
- 데이터 세트에 대해 feature X에 대해 Target Y 값의 관계를 단순 선형 회귀 직선형으로 표현한 것보다 다항 회귀 곡선의 예측 성능이 더 높음



- 사이킷런은 다항회귀를 위한 클래스 명시적 제공하지 않음
- 비선형 함수를 선형 모델에 적용시키는 방법 사용하여 구현

- PolynomialFeatures 클래스를 통해 feature를 Polynomial(다항식) feature로 변환
- PolynomialFeatures 클래스는 degree 파라미터 통해 입력받은 단항식 feature를 degree에 해당하는 다항식 feature로 변환

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np

# 다항식으로 변환한 단항식 생성, [[0,1],[2,3]]의 2X2 행렬 생성
X = np.arange(4).reshape(2,2)
print('일차 단항식 계수 feature:\n',X )

# degree = 2 인 2차 다항식으로 변환하기 위해 PolynomialFeatures를 이용하여 변환
poly = PolynomialFeatures(degree=2)
poly.fit(X)
poly_ftr = poly.transform(X)
print('변환된 2차 다항식 계수 feature:\n', poly_ftr)
```

일차 단항식 계수 feature:

```
[[0 1]
 [2 3]]
```

변환된 2차 다항식 계수 feature:

```
[[1. 0. 1. 0. 0. 1.]
 [1. 2. 3. 4. 6. 9.]]
```

- 이렇게 변환된 Polynomial feature에 선형 회귀를 적용해 다항 회귀를 구현
- 3차 다항 회귀 함수식을 PolynomialFeatures와 LinearRegression 클래스를 이용해 유도
- 3차 다항 회귀 함수 임의 설정, 이의 회귀 계수 예측

```
def polynomial_func(X):
    y = 1 + 2*X[:,0] + 3*X[:,0]**2 + 4*X[:,1]**3
    print(X[:, 0])
    print(X[:, 1])
    return y

X = np.arange(0,4).reshape(2,2)

print('일차 단항식 계수 feature: \n',X)
y = polynomial_func(X)
print('삼차 다항식 결정값: \n', y)

일차 단항식 계수 feature:
[[0 1]
 [2 3]]
[0 2]
[1 3]
삼차 다항식 결정값:
[ 5 125]
```

```

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
import numpy as np

def polynomial_func(X):
    y = 1 + 2*X[:,0] + 3*X[:,0]**2 + 4*X[:,1]**3
    return y

# Pipeline 객체로 Streamline 하게 Polynomial Feature변환과 Linear Regression을 연결
model = Pipeline([('poly', PolynomialFeatures(degree=3)),
                  ('linear', LinearRegression())])
X = np.arange(4).reshape(2,2)
y = polynomial_func(X)

model = model.fit(X, y)
print('Polynomial 회귀 계수\n', np.round(model.named_steps['linear'].coef_, 2))

Polynomial 회귀 계수
[0.   0.18 0.18 0.36 0.54 0.72 0.72 1.08 1.62 2.34]

```

다항 회귀를 이용한 과소적합 및 과적합 이해

다항회귀

- feature의 직선적 관계가 아닌 복잡한 다항 관계를 모델링 가능
- 다항 회귀의 차수(degree) 높일수록 학습 데이터에만 너무 맞춘 학습
- 정작 테스트 데이터 환경에서는 오히려 예측 정확도 떨어짐
- ⇒ 차수 높아질수록 과적합 문제 크게 발생
- 사이킷런 홈페이지의 예시

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
%matplotlib inline

# 임의의 값으로 구성된 X값에 대해 코사인 변환 값을 반환.
def true_fun(X):
    return np.cos(1.5 * np.pi * X)

# X는 0부터 1까지 30개의 임의의 값을 순서대로 샘플링한 데이터입니다.
np.random.seed(0)
n_samples = 30
X = np.sort(np.random.rand(n_samples))

# y 값은 코사인 기반의 true_fun()에서 약간의 노이즈 변동 값을 더한 값입니다.
y = true_fun(X) + np.random.randn(n_samples) * 0.1

```

```

plt.figure(figsize=(14, 5))
degrees = [1, 4, 15]

# 다항 회귀의 차수(degree)를 1, 4, 15로 각각 변화시키면서 비교합니다.
for i in range(len(degrees)):
    ax = plt.subplot(1, len(degrees), i + 1)
    plt.setp(ax, xticks=(), yticks=())

    # 개별 degree별로 Polynomial 변환합니다.
    polynomial_features = PolynomialFeatures(degree=degrees[i], include_bias=False)
    linear_regression = LinearRegression()
    pipeline = Pipeline([("polynomial_features", polynomial_features),
                          ("linear_regression", linear_regression)])
    pipeline.fit(X.reshape(-1, 1), y)

    # 교차 검증으로 다항 회귀를 평가합니다.
    scores = cross_val_score(pipeline, X.reshape(-1, 1), y, scoring="neg_mean_squared_error", cv=10)
    # Pipeline을 구성하는 세부 객체를 접근하는 named_steps['객체명']을 이용해 회귀계수 추출
    coefficients = pipeline.named_steps['linear_regression'].coef_
    print('\nDegree {0} 회귀 계수는 {1} 입니다.'.format(degrees[i], np.round(coefficients, 2)))
    print('Degree {0} MSE 는 {1} 입니다.'.format(degrees[i], -1*np.mean(scores)))

    # 0 부터 1까지 테스트 데이터 세트를 100개로 나눠 예측을 수행합니다.
    # 테스트 데이터 세트에 회귀 예측을 수행하고 예측 곡선과 실제 곡선을 그려서 비교합니다.
    X_test = np.linspace(0, 1, 100)
    # 예측값 곡선
    plt.plot(X_test, pipeline.predict(X_test[:, np.newaxis]), label="Model")
    # 실제 값 곡선
    plt.plot(X_test, true_fun(X_test), '-', label="True function")
    plt.scatter(X, y, edgecolor='b', s=20, label="Samples")
    plt.xlabel("x"); plt.ylabel("y"); plt.xlim((0, 1)); plt.ylim((-2, 2)); plt.legend(loc="best")
    plt.title("Degree {0} \nMSE = {:.2e} (+/- {:.2e})".format(degrees[i], -scores.mean(), scores.std()))

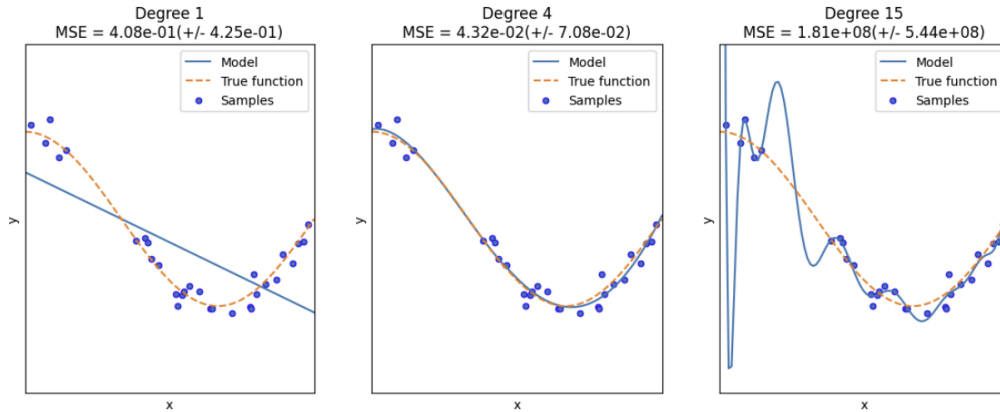
plt.show()

```

Degree 1 회귀 계수는 [-1.61] 입니다.
Degree 1 MSE 는 0.4077289625098685 입니다.

Degree 4 회귀 계수는 [0.47 -17.79 23.59 -7.26] 입니다.
Degree 4 MSE 는 0.04320874987232064 입니다.

Degree 15 회귀 계수는 [-2.98293000e+03 1.03899390e+05 -1.87416123e+06 2.03716219e+07
-1.44873283e+08 7.09315363e+08 -2.47065792e+09 6.24561050e+09
-1.15676510e+10 1.56894936e+10 -1.54006023e+10 1.06457264e+10
-4.91377530e+09 1.35919645e+09 -1.70380786e+08] 입니다.
Degree 15 MSE 는 181238256.56423894 입니다.

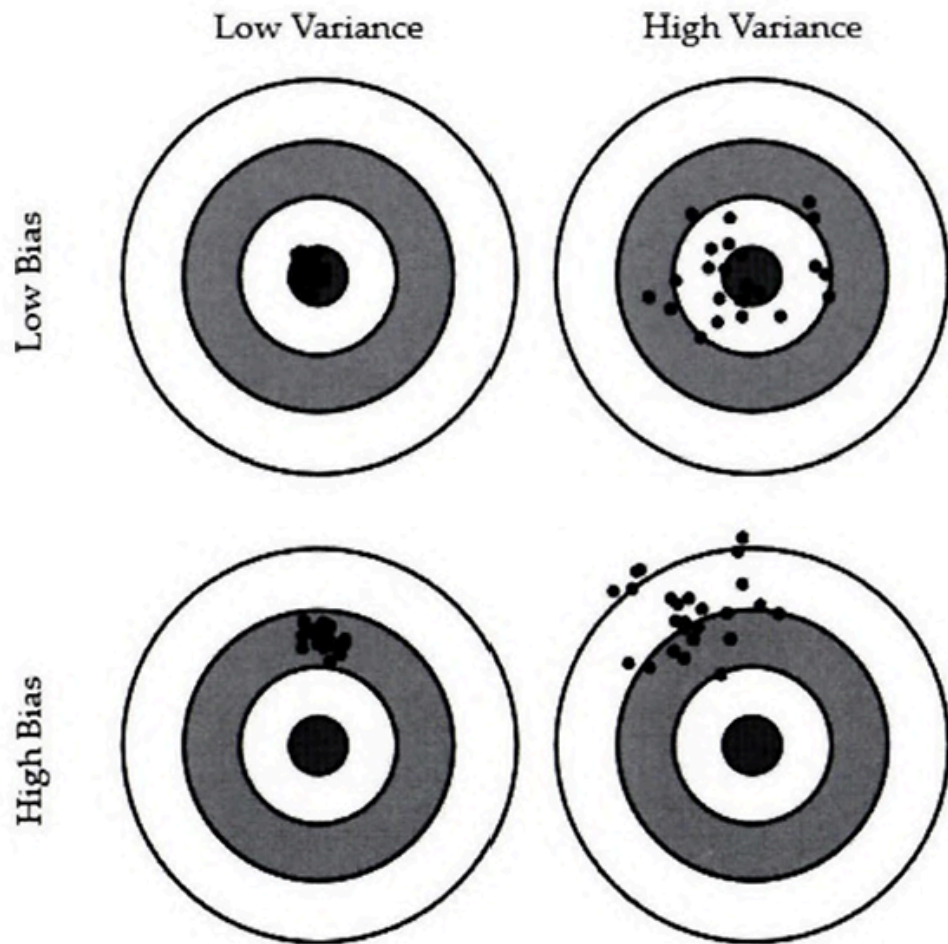


- 실선 예측 곡선은 다항회귀 예측 곡선
- 점선 곡선 : 실제 데이터 세트 곡선

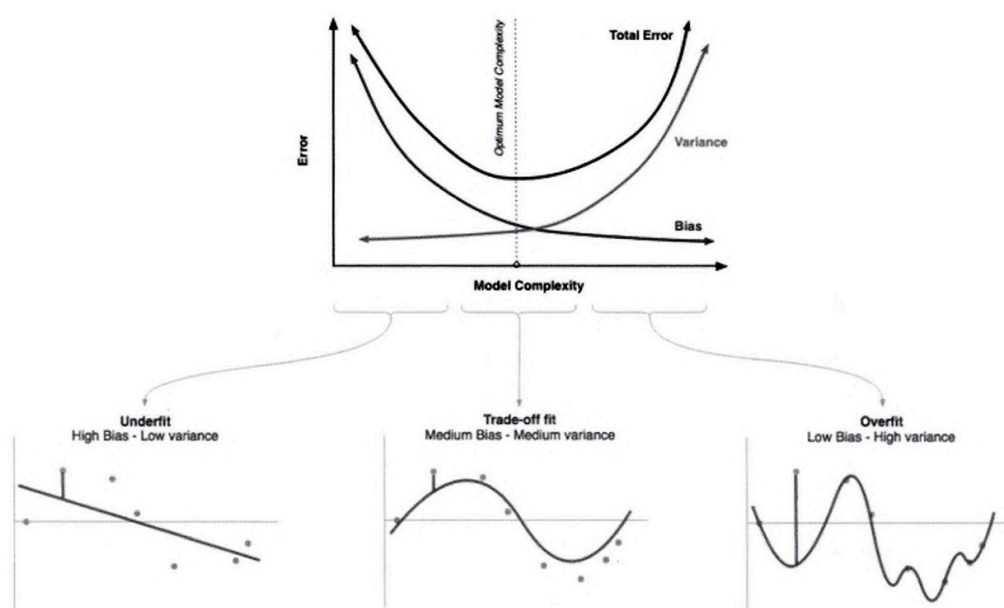
- 맨 왼쪽의 Degree 1 예측 곡선은 단순한 직선으로서 단순 선형 회귀와 똑같습니다. 실제 데이터 세트인 코사인 데이터 세트를 직선으로 예측하기에는 너무 단순해 보입니다. 예측 곡선이 학습 데이터의 패턴을 제대로 반영하지 못하고 있는 과소 적합 모델이 되었습니다. MSE 값은 약 0.41입니다.
- 가운데 Degree 4 예측 곡선은 실제 데이터 세트와 유사한 모습입니다. 변동하는 잡음까지 예측하지는 못했지만, 학습 데이터 세트를 비교적 잘 반영해 코사인 곡선 기반으로 테스트 데이터를 잘 예측한 곡선을 가진 모델이 되었습니다. MSE 값은 약 0.04로 가장 뛰어난 예측 성능을 나타내고 있습니다.
- 맨 오른쪽 Degree 15 예측 곡선은 MSE 값이 182581084.83이 될 정도로 어처구니없는 오류 값이 발생했습니다(물론 과적합을 강조하기 위해 Degree를 매우 높은 차수인 15로 설정한 결과입니다). 예측 곡선을 보면 데이터 세트의 변동 잡음 값까지 지나치게 반영한 결과, 예측 곡선이 학습 데이터 세트만 정확히 예측하고, 테스트 값의 실제 곡선과는 완전히 다른 형태의 예측 곡선이 만들어졌습니다. 결과적으로 학습 데이터에 너무 충실하게 맞춘 과적합이 심한 모델이 되었고 어이없는 수준의 높은 MSE 값이 나왔습니다.

편향-분산 트레이드오프(Bias-Variance Trade Off)

- 머신러닝이 극복해야 할 가장 중요한 이슈 중 하나
- 고편향 : 지나치게 치우친 단순화된 모델



- 일반적으로 편향과 분산은 한쪽이 높으면 한쪽이 낮아지는 경향



- 편향과 분산의 관계에 따른 전체 오류 값 변화
- 높은 편향/낮은 분산에서 과소적합되기 쉬움
- 낮은 편향/ 높은 분산에서 과적합되기 쉬움

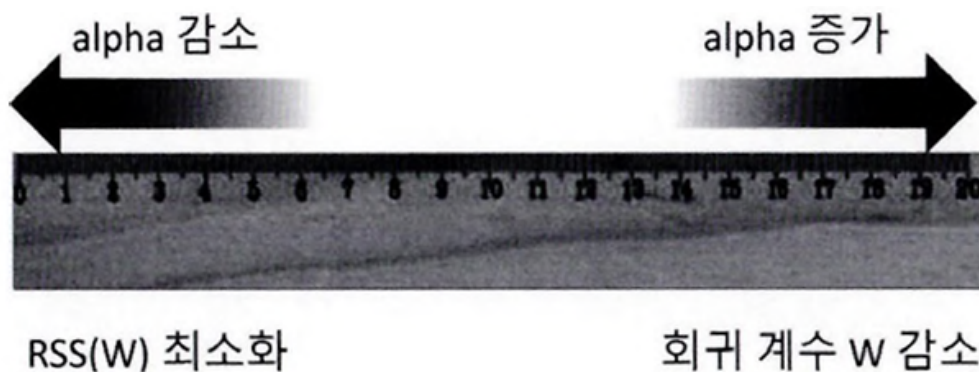
06 규제 선형 모델 - 릿지, 라쏘, 엘라스틱넷

규제 선형 모델의 개요

- 회귀 모델은 적절히 데이터에 적합하면서도 회귀 계수가 기하급수적으로 커지는 것 제어해야 함
- RSS 최소화에만 집중하면 학습 데이터에 지나치게 맞춤, 회귀 계수 쉽게 커짐
- → 변동성 심해 오히려 예측 성능 저하되기 쉬움

$$\text{비용 함수 목표} = \text{Min}(\text{RSS}(W) + \alpha * \|W\|_2^2)$$

- alpha는 학습 데이터 적합 정도와 회귀 계수 값의 크기 제어를 수행하는 튜닝 파라미터
 - alpha = 0인 경우는 W가 커도 $\alpha * \|W\|_2^2$ 가 0이 되어 비용 함수는 $\text{Min}(\text{RSS}(W))$
 - alpha = 무한대인 경우 $\alpha * \|W\|_2^2$ 도 무한대가 되므로 비용 함수는 W를 0에 가깝게 최소화 해야 함.



- L2 규제

- $\alpha * ||W||_2^2$ 처럼 W의 제곱에 대해 패널티 부여하는 방식
- '릿지' 회귀
- L1 규제
 - $\alpha * ||W||_1$ 처럼 절댓값에 대해 패널티 부여
 - '라쏘' 회귀
 - 영향력이 크지 않은 회귀 계수 값을 0으로 변환

엘라스틱넷 회귀

- L2 규제와 L1 규제를 결합한 회귀
- 라쏘 회귀가 서로 상관관계가 높은 feature들의 경우에 중요 feature만을 선택하고 다른 feature들은 모두 회귀 계수를 0으로 만드는 성향 강함
- 이런 성향으로 alpha 값에 따라 회귀 계수의 값이 급격히 변동 가능
- → 완화하기 위해 L2 규제를 라쏘 회귀에 추가한 것
- 시간 오래 걸린다는 단점 있음
- 사이킷런은 ElasticNet 클래스 통해서 엘라스틱넷 회귀 구현
- 생성 파라미터는 alpha와 l1_ratio
- l1_ratio=0이면 a=0이므로 L2 규제와 동일, l1_ratio=1이면 b=0이므로 L1 규제와 동일

선형 회귀 모델을 위한 데이터 변환

- feature와 타깃값 간에 선형 관계 있다고 가정하고 최적의 선형 함수 찾아내 결괏값 예측
- feature값, target값 정규 분포 형태를 매우 선호
- 일반적으로 feature 데이터 세트와 target 데이터 세트에 이러한 스케일링/정규화 작업 수행하는 방법 조금 다름
- 사이킷런 이용해 feature 데이터 세트에 적용하는 변환 작업 방법

1. StandardScaler 클래스를 이용해 평균이 0, 분산이 1인 표준 정규 분포를 가진 데이터 세트로 변환하거나 MinMaxScaler 클래스를 이용해 최소값이 0이고 최대값이 1인 값으로 정규화를 수행합니다.
2. 스케일링/정규화를 수행한 데이터 세트에 다시 다항 특성을 적용하여 변환하는 방법입니다. 보통 1번 방법을 통해 예측 성능에 향상이 없을 경우 이와 같은 방법을 적용합니다.
3. 원래 값에 log 함수를 적용하면 보다 정규 분포에 가까운 형태로 값이 분포됩니다. 이러한 변환을 로그 변환(Log Transformation)이라고 부릅니다. 로그 변환은 매우 유용한 변환이며, 실제로 선형 회귀에서는 앞에서 소개한 1, 2 번 방법보다 로그 변환이 훨씬 많이 사용되는 변환 방법입니다. 왜냐하면 1번 방법의 경우 예측 성능 향상을 크게 기대하기 어려운 경우가 많으며 2번 방법의 경우 피처의 개수가 매우 많을 경우에는 다항 변환으로 생성되는 피처의 개수가 기하급수로 늘어나서 과적합의 이슈가 발생할 수 있기 때문입니다.

- 타깃값 경우 일반적으로 로그 변환 적용

-

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, PolynomialFeatures

# method는 표준 정규 분포 변환(Standard), 최대값/최소값 정규화(MinMax), 로그변환(Log) 결정
# p_degree는 다항식 특성을 추가할 때 적용. p_degree는 2이상 부여하지 않음.
def get_scaled_data(method='None', p_degree=None, input_data=None):
    if method == 'Standard':
        scaled_data = StandardScaler().fit_transform(input_data)
    elif method == 'MinMax':
        scaled_data = MinMaxScaler().fit_transform(input_data)
    elif method == 'Log':
        scaled_data = np.log1p(input_data)
    else:
        scaled_data = input_data

    if p_degree != None:
        scaled_data = PolynomialFeatures(degree=p_degree,
                                         include_bias=False).fit_transform(scaled_data)

    return scaled_data
```

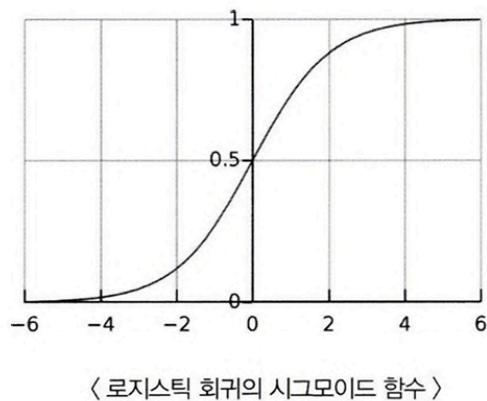
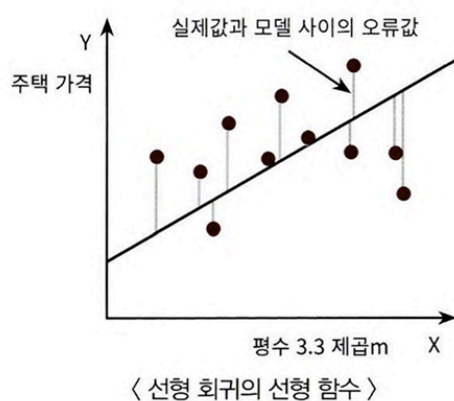
```
alphas = [0.1, 1, 10, 100]
#변환 방법은 모두 6개, 원본 그대로, 표준정규분포, 표준정규분포+다항식 특성
# 최대/최소 정규화, 최대/최소 정규화+다항식 특성, 로그변환
scale_methods=[(None, None), ('Standard', None), ('Standard', 2),
                ('MinMax', None), ('MinMax', 2), ('Log', None)]
for scale_method in scale_methods:
    X_data_scaled = get_scaled_data(method=scale_method[0], p_degree=scale_method[1],
                                     input_data=X_data)
    print(X_data_scaled.shape, X_data.shape)
    print('\n## 변환 유형:{0}, Polynomial Degree:{1}'.format(scale_method[0], scale_method[1]))
    get_linear_reg_eval('Ridge', params=alphas, X_data_n=X_data_scaled,
                        y_target_n=y_target, verbose=False, return_coeff=False)
```

변환 유형	alpha 값			
	alpha=0.1	alpha=1	alpha=10	alpha=100
원본 데이터	5.788	5.653	5.518	5.330
표준 정규 분포	5.826	5.803	5.637	5.421
표준 정규 분포 + 2차 다항식	8.827	6.871	5.485	4.634
최솟값/최댓값 정규화	5.764	5.465	5.754	7.635
최솟값/최댓값 정규화 + 2차 다항식	5.298	4.323	5.185	6.538
로그 변환	4.770	4.676	4.836	6.241

- 일반적으로 선형 회귀를 적용하려는 데이터 세트에 데이터 값의 분포가 심하게 왜곡되어 있을 경우, 로그 변환을 적용하는 것이 좋은 결과 기대 쉬움

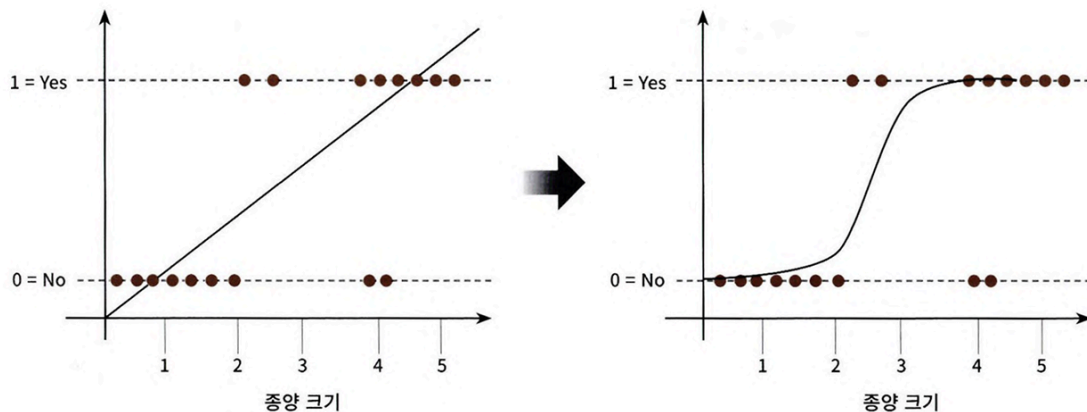
07 로지스틱 회귀

- 선형 회귀 방식을 분류에 적용한 알고리즘
- 학습을 통해 선형 함수의 회귀 최적선을 찾는 것이 아니라 시그모이드 함수 최적선을 찾고 이 시그모이드 함수의 반환 값을 확률로 간주해 확률에 따라 분류 결정



- 시그모이드 함수는 x가 아무리 증/감소해도 0과 1 사이 값을 반환
- 종양 분류 문제에 적용

- yes(=1), no(=0)



- 사이킷런은 로지스틱 회귀를 위해 LogisticRegression 클래스 제공
- 경사 하강법 외에 다양한 최적화 방법 선택 가능
 - lbfgs: 사이킷런 버전 0.22부터 solver의 기본 설정값입니다. 메모리 공간을 절약할 수 있고, CPU 코어 수가 많다면 최적화를 병렬로 수행할 수 있습니다.
 - liblinear: 사이킷런 버전 0.21까지에서 solver의 기본 설정값입니다. 다차원이고 작은 데이터 세트에서 효과적으로 동작하지만 국소 최적화(Local Minimum)에 이슈가 있고, 병렬로 최적화할 수 없습니다.
 - newton-cg: 좀 더 정교한 최적화를 가능하게 하지만, 대용량의 데이터에서 속도가 많이 느려집니다.
 - sag: Stochastic Average Gradient로서 경사 하강법 기반의 최적화를 적용합니다. 대용량의 데이터에서 빠르게 최적화합니다.
 - saga: sag와 유사한 최적화 방식이며 L1 정규화를 가능하게 해줍니다.

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression

cancer = load_breast_cancer()

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# StandardScaler( )로 평균이 0, 분산 1로 데이터 분포도 변환
scaler = StandardScaler()
data_scaled = scaler.fit_transform(cancer.data)

X_train, X_test, y_train, y_test = train_test_split(data_scaled, cancer.target, test_size=0.3, random_state=
```

- solver 값을 lbfgs로 설정하고 성능 확인

```
from sklearn.metrics import accuracy_score, roc_auc_score

# 로지스틱 회귀를 이용하여 학습 및 예측 수행.
# solver인자값을 생성자로 입력하지 않으면 solver='lbfgs'
lr_clf = LogisticRegression() # solver='lbfgs'
lr_clf.fit(X_train, y_train)
lr_preds = lr_clf.predict(X_test)
lr_preds_proba = lr_clf.predict_proba(X_test)[: , 1]

# accuracy와 roc_auc 측정
print('accuracy: {0:.3f}, roc_auc:{1:.3f}'.format(accuracy_score(y_test, lr_preds),
                                                    roc_auc_score(y_test , lr_preds_proba)))

accuracy: 0.977, roc_auc:0.995
```

- 서로 다른 solver 값으로 성능 평가
- max_iter값을 600으로 설정

```
solvers = ['lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga']
# 여러개의 solver값 별로 LogisticRegression 학습 후 성능 평가
for solver in solvers:
    lr_clf = LogisticRegression(solver=solver, max_iter=600)
    lr_clf.fit(X_train, y_train)
    lr_preds = lr_clf.predict(X_test)
    lr_preds_proba = lr_clf.predict_proba(X_test)[: , 1]

    # accuracy와 roc_auc 측정
    print('solver:{0}, accuracy: {1:.3f}, roc_auc:{2:.3f}'.format(solver,
                                                                    accuracy_score(y_test, lr_preds),
                                                                    roc_auc_score(y_test , lr_preds_proba)))

solver:lbfgs, accuracy: 0.977, roc_auc:0.995
solver:liblinear, accuracy: 0.982, roc_auc:0.995
solver:newton-cg, accuracy: 0.977, roc_auc:0.995
solver:sag, accuracy: 0.982, roc_auc:0.995
solver:saga, accuracy: 0.982, roc_auc:0.995
```

- GridSearchCV 이용해 solver, penalty, C 최적화

```

from sklearn.model_selection import GridSearchCV

params={'solver':['liblinear', 'lbfgs'],
        'penalty':['l2', 'l1'],
        'C':[0.01, 0.1, 1, 5, 10]}

lr_clf = LogisticRegression()

grid_clf = GridSearchCV(lr_clf, param_grid=params, scoring='accuracy', cv=3 )
grid_clf.fit(data_scaled, cancer.target)
print('최적 하이퍼 파라미터:{0}, 최적 평균 정확도:{1:.3f}'.format(grid_clf.best_params_,
                                                                grid_clf.best_score_))

```

최적 하이퍼 파라미터:{'C': 0.1, 'penalty': 'l2', 'solver': 'liblinear'}, 최적 평균 정확도:0.979
/usr/local/lib/python3.11/dist-packages/sklearn/model_selection/_validation.py:528: FitFailedWarning:
15 fits failed out of a total of 60.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:

15 fits failed with the following error:
Traceback (most recent call last):
File "/usr/local/lib/python3.11/dist-packages/sklearn/model_selection/_validation.py", line 866, in _fit_and_estimator.fit(X_train, y_train, **fit_params)
File "/usr/local/lib/python3.11/dist-packages/sklearn/base.py", line 1389, in wrapper
return fit_method(estimator, *args, **kwargs)
~~~~~  
File "/usr/local/lib/python3.11/dist-packages/sklearn/linear\_model/\_logistic.py", line 1193, in fit  
solver = \_check\_solver(self.solver, self.penalty, self.dual)  
~~~~~  
File "/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py", line 63, in _check_solver
raise ValueError(
ValueError: Solver lbfgs supports only 'l2' or None penalties, got l1 penalty.

```

warnings.warn(some_fits_failed_message, FitFailedWarning)
/usr/local/lib/python3.11/dist-packages/sklearn/model_selection/_search.py:1108: UserWarning: One or more of th
0.96131997      nan 0.97539218 0.97539218 0.96660169      nan
0.97011974 0.96836536 0.96662025      nan 0.96661097 0.96661097
0.96134781      nan]
warnings.warn(

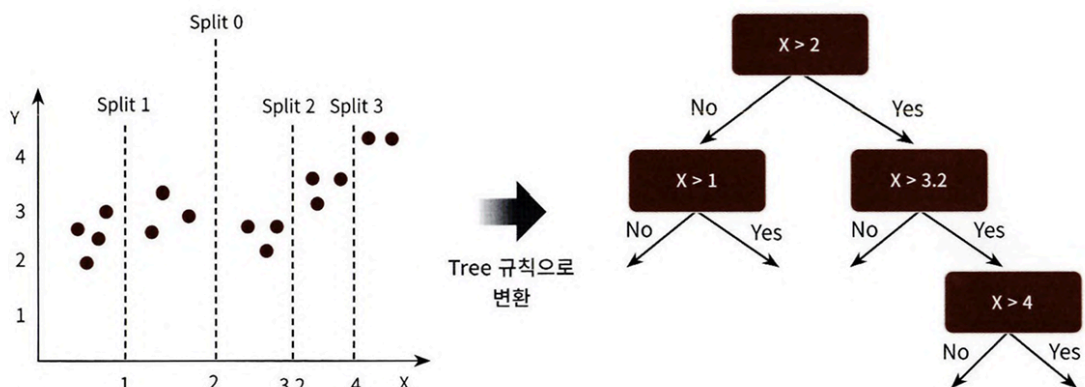
```

- logistic regression은 가볍고 빠르지만 이진 분류 예측 성능도 뛰어남

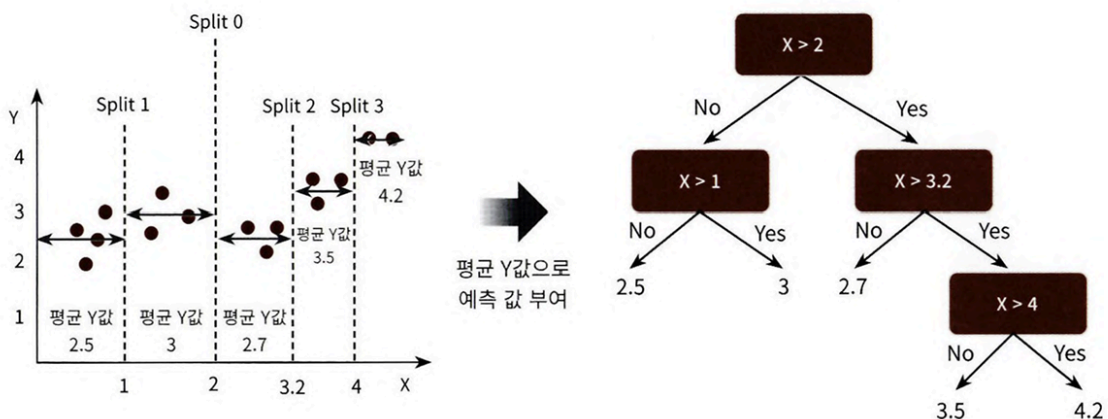
08 회귀 트리

- 회귀 함수 기반이 아닌 결정 트리와 같이 트리를 기반으로 하는 회귀 방식
- 회귀를 위한 트리를 생성하고 회귀 예측
- 분류 트리가 특정 클래스 레이블을 결정하는 것과는 달리 회귀 트리는 리프 노드에 속한 데이터 값의 평균값을 구해 회귀 예측값을 계산
- feature X 하나, 결정값 Y

- X feature를 결정 트리 기반으로 분할하면 X값의 균일도를 반영한 지니 계수에 따라



- 리프 노드 생성 기준에 부합하는 트리 분할 완료 되면, 리프 노드에 소속된 데이터 값의 평균값을 구해 최종적으로 리프 노드에 결정 값 할당



- 사이킷런에서 결정트리, 랜덤포레스트, GBM에서 CART 기반의 회귀 수행을 알 수 있는 Estimator 클래스 제공

알고리즘	회귀 Estimator 클래스	분류 Estimator 클래스
Decision Tree	DecisionTreeRegressor	DecisionTreeClassifier
Gradient Boosting	GradientBoostingRegressor	GradientBoostingClassifier
XGBoost	XGBRegressor	XGBClassifier
LightGBM	LGBMRegressor	LGBMClassifier

```

from sklearn.datasets import load_boston
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestRegressor
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore') #사이킷런 1.2 부터는 보스턴 주택가격 데이터가 없어진다는 warning 메시지 출력

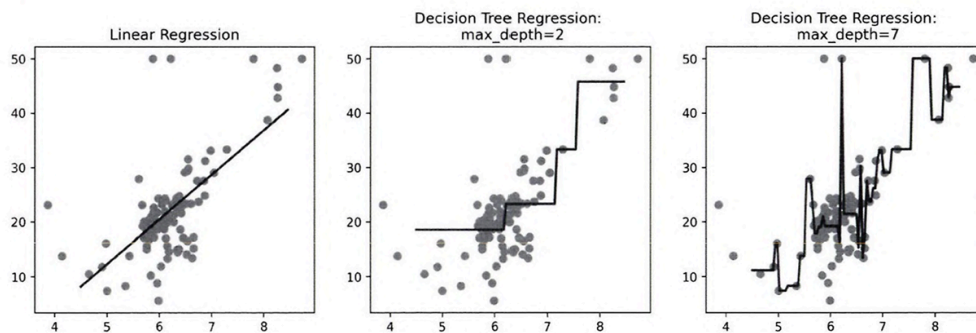
# 보스턴 데이터 세트 로드
boston = load_boston()
bostonDF = pd.DataFrame(boston.data, columns = boston.feature_names)

bostonDF['PRICE'] = boston.target
y_target = bostonDF['PRICE']
X_data = bostonDF.drop(['PRICE'], axis=1,inplace=False)

rf = RandomForestRegressor(random_state=0, n_estimators=1000)
neg_mse_scores = cross_val_score(rf, X_data, y_target, scoring="neg_mean_squared_error", cv = 5)
rmse_scores = np.sqrt(-1 * neg_mse_scores)
avg_rmse = np.mean(rmse_scores)

print(' 5 교차 검증의 개별 Negative MSE scores: ', np.round(neg_mse_scores, 2))
print(' 5 교차 검증의 개별 RMSE scores : ', np.round(rmse_scores, 2))
print(' 5 교차 검증의 평균 RMSE : {0:.3f} '.format(avg_rmse))

```



- 회귀 트리의 경우 분할되는 데이터 지점에 따라 브랜치를 만들면서 계단 형태로 회귀선 만들
- DecisionTreeRegressor의 max_depth=7인 경우 학습 데이터 세트의 이상치 데이터도 학습하면서 복잡한 계단 형태의 회귀선 만들어 과적합되기 쉬운 모델이 됨